

NATIONAL UNIVERSITY OF SINGAPORE

EE4705: Human Robot Interaction

MiniLab 1.3: Command Your Robot Dog: An LLM + YOLO Powered Quadruped

(15%)

I. Instruction

1. This mini-lab is a joint work to show your collaborative effort, and at the same time, each individual can show his/her talent for individual portions as your choices and declaration. Students are required to work together to complete it with 3 students (2 students in special case) as a group.
2. Zip all files into one compressed file. Name your file as minilab_1.3 with your group index and submit it into the folder Student Submission: MiniLab 1.3 under Canvas.
Example: if your group index is 1, the file name must be minilab_1.3_group_1.zip.
3. Put your names and matriculation numbers on the cover page. There should only be one combined report submitted per group. Recommended pages: 8–15 pages, 1.5-line spacing, 12-point “Times New Roman” font, 1-inch margins. The zipped file must include: (a) the report in PDF; (b) all your source code with install and run instructions; (c) your scene/object files (MJCF or equivalent, plus object meshes or their download links) and your LLM prompt files; and (d) three demo videos — Video_Task2 (platform, scene, camera view and motion skills), Video_Task3 (typed English motion command executed autonomously) and Video_Task4 (typed English object-search command, YOLO-guided navigation connected to Task 3). In Video_Task3 and Video_Task4 the terminal window must remain visible throughout. Include proper citations for all sources.
4. The submission deadline is on **4th Oct 2026**; any submissions after that will be subject to the late submission policy.
5. Group members can choose how to split the tasks amongst themselves; however, the tasks should be split evenly to ensure that each member's contribution can be adequately appraised. A suggested split is Task 1 (ALL), Task 2 (Student A), Task 3 (Student B), Task 4 (Student C) and Task 5 (ALL). In a 2-member group, Student A leads Task 2, Student B leads Task 3, and both share Task 4. Each student must have their contribution clearly stated in the report

and marked in the source files. Identical submissions will be subject to penalty and/or disciplinary actions of the university.

6. Use of AI coding assistants (e.g., ChatGPT, Claude, GitHub Copilot, Cursor) is **allowed and strongly encouraged** in this mini-lab to accelerate development (“vibe coding”). However:
(a) you must fully understand every line you submit and be able to explain it during assessment;
(b) your report must include a short AI Usage Declaration stating which tools you used and for what; (c) you are graded on the working system, your design decisions, and your analysis — not on how the code was typed. Blindly submitted, non-functional AI-generated code will be penalized.
7. The whole mini-lab runs on ordinary laptops without a dedicated GPU: the MuJoCo simulation, the pre-trained walking policy (ONNX) and the YOLO detector all run locally on CPU. Only the LLM runs in the cloud, and only text-only LLM calls are needed, which are much cheaper than image calls. With the free tiers (Qwen via Alibaba Cloud Singapore, Gemini) the mini-lab can be completed at zero API cost; a paid OpenAI key typically costs well under US\$1–2 in total. Budget under US\$5.
8. Any queries, feel free to contact GAs:
 - Mr. Huang Dong, dong.huang@u.nus.edu,
 - Mr. Ying Zhuohang, E1554788@u.nus.edu, and
 - Mr. Zhang Aoqian, aoqian@u.nus.edu.

II. Objective

A simulated quadruped (“robot dog”) with a pre-trained reinforcement-learning walking policy must execute typed English commands: first motion sequences (“walk forward for three seconds, then turn back”), then object-directed navigation (“go to the green chair”) using its onboard camera and a YOLO detector. The closed loop of command → LLM parsing → autonomous motion → perception → confirmation is the essence of this mini-lab. Task 3 and Task 4 must use the same platform, scene and motion skills built in Task 2. After a valid command is accepted, the robot must commence autonomous motion and continue until every commanded action is completed, in order, or the mission fails. The objectives of this mini-lab are:

1. To compare approaches for connecting language models to robot control and to explain the two-rate architecture (slow LLM planner over a fast local control loop) used in this mini-lab.

2. To set up a quadruped simulation with a learned locomotion policy, an onboard camera whose image is captured at each control frame, a scene of detectable objects, and reusable motion skills (timed moves and closed-loop turns).
3. In Task 3, to accept a typed English command that describes one or more motions and to execute them in order by fully autonomous motion, using an LLM parser with structured (JSON) output.
4. In Task 4, to implement camera-based YOLO object detection with color grounding, and to connect it to Task 3 so that an object-search command results in autonomous search and approach.
5. To submit a group report and three demonstration videos that allow the system to be reproduced and assessed.
6. To keep all operator commands in English only; non-English commands must be rejected or must trigger a request for an English command.

The reference commands. Throughout this document we use the following interaction as the reference scenario your final system must support. Several everyday objects (e.g., chairs, a ball, a stop sign) are placed by you in the simulated world, including at least two objects of the same class in different colors:

```
User: walk forward for three seconds, then turn back
Robot: [CMD] actions=move(vx=0.8, 3.0 s), turn(180 deg) n=2
       [EXEC] action=1/2 move vx=0.8 vy=0.0 wz=0.0 t=3.0 s
       [TURN] target=180.0 deg final_error=1.8 deg
       [DONE] actions=2 t=9.4 s
User: go to the green chair
Robot: [CMD] goto_object class=chair color=green
       [SEARCH] target not visible, rotating
       [DETECT] class=chair color=green conf=0.72 bbox=[212,140,318,352]
       [FOUND] class=chair color=green t=14.2 s d=0.61 m
       [MISSION] status=SUCCESS
```

Note: each group only needs to submit one completed system; members' contributions need to be marked in the report and in the source files.

III. Tasks

The mini-lab consists of five tasks. Figure 1 shows the reference architecture that Tasks 2–4 build up together: Task 2 provides the platform, the camera, the scene and the motion skills; Task 3 provides the LLM command parser and the chat interface; Task 4 provides YOLO

perception and the object-approach controller. You may deviate from this architecture if you justify your design in the report. Task 3 and Task 4 must run on the Task 2 platform and scene; do not substitute another robot or scene for the later tasks.

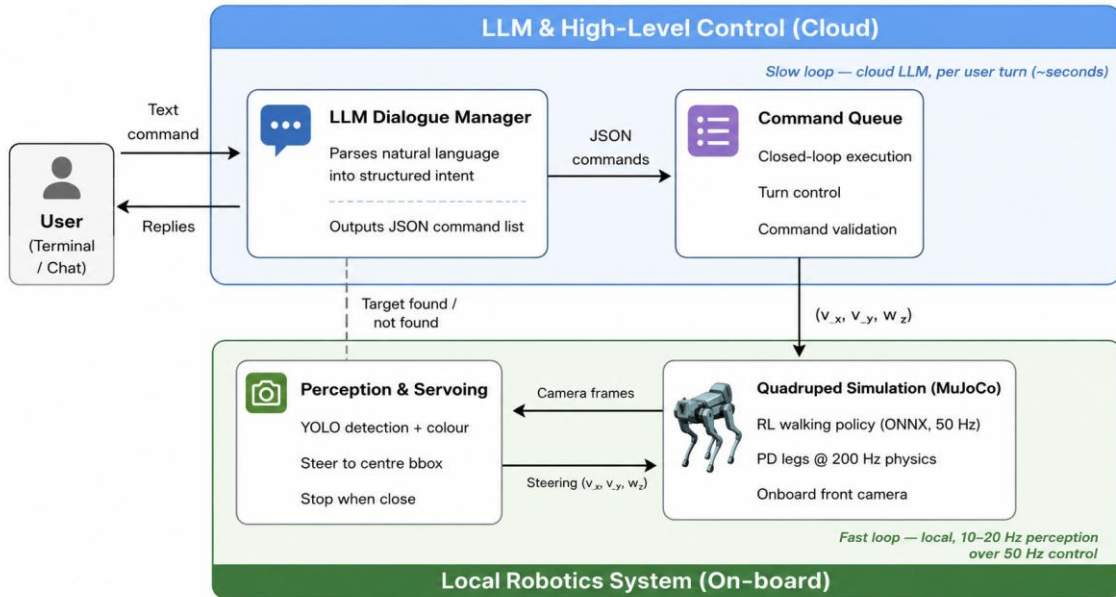


Fig. 1. Reference architecture: a slow cloud LLM planner (Task 3) on top of a fast local perception-and-control loop (Tasks 2 and 4). Component names refer to the example platform; the same architecture applies to any platform you choose.

Task 1: Language Models for Robot Control and Learned Legged Locomotion (ALL)

- i. Compare at least three approaches for connecting language models to robots. The comparison must include structured-output command parsing (JSON mode / function calling, as used in this mini-lab) and at least two of the following: affordance-grounded planning (SayCan), LLM-generated robot code (Code as Policies), LLM agents over robot middleware (ROSGPT, NASA JPL ROSA), vision-language-action models (RT-2), or VLM-based visual grounding.
- ii. For each approach, state the core idea, what the model outputs (text, JSON, code, or actions), its latency and cost profile, and the application scenarios in which it is suitable (for example: tabletop manipulation, indoor navigation, multi-step household tasks, or safety-critical settings).
- iii. Briefly explain how the quadruped's walking policy is obtained (reinforcement learning in simulation with a velocity-command interface, and sim-to-sim/sim-to-

real transfer), and why this mini-lab keeps the LLM out of the fast control loop — the two-rate architecture in Figure 1 — with reference to LLM latency.

- iv. Write this discussion in the group report (about one page). Cite at least two references. No software implementation is required for Task 1.

Task 2: Platform Setup, Scene Building and Motion Skills (Student A)

- i. Choose your platform from Section V and set it up. The recommended default is the example repository (https://github.com/aoqianz/quadruped_mujoco): clone it, install it, and run the demo `eg/play.py` in both display modes — the native MuJoCo viewer and the browser control panel (`--gui`); drive the robot with the keyboard (W/S/A/D/Q/E), switch between at least 3 of the bundled terrain maps, and view the three onboard cameras. In the report, explain the control pipeline with a labeled block diagram — for the example repository: the velocity command interface (v_x , v_y , w_z , body height), the 46-dimensional observation and its 6-frame history, the ONNX policy running at 50 Hz over 200 Hz physics (control decimation 4), the PD torque loop, and why the rear-leg joint remapping between IsaacGym and MuJoCo ordering is needed. For another platform, cover the equivalent chain from motion command to actuation.
- ii. Camera pipeline. Render the onboard front camera offscreen and obtain an RGB image at each perception frame (10–20 Hz is sufficient and recommended; state your rate in the report and justify it). Expose the latest frame through a small function or class that other members can call, and save annotated frames or a short clip as evidence. This is the only image source Task 4 may use.
- iii. Scene building. Create your own scene file (MJCF for the example platform) with at least 3 objects from at least 2 different COCO classes that YOLO can detect on rendered images (e.g., chair meshes, a sports ball, a stop sign), including at least two objects of the same class in different colors (e.g., a green chair and a red chair). Verify detection early: write a small script that renders frames of your scene from a robot-height viewpoint, runs YOLO on them, and draws the boxes; include one such screenshot in the report. A plain colored box will not be detected as a “chair”; free

meshes can be found on Objaverse or Sketchfab. Record the object positions in a configuration file — they are needed for the distance logging in Task 4.

- iv. Motion skills. Implement a small motion-skills API that Tasks 3 and 4 will call: (a) a timed move, `move(vx, vy, wz, duration)`, served by a queue of timed velocity commands that replaces the platform's keyboard command source (in the example repository, `get_commands()` in `eg/play.py`) while keeping the rest of the control loop unchanged; and (b) a closed-loop turn, `turn(angle_deg)`, that uses the robot's actual yaw read from the simulation state (for MuJoCo, `mj_data.qpos`), not open-loop timing, and prints `[TURN] target=... final_error=...`. Test both skills from keyboard shortcuts. Show in the report (a short table or plot) why timed open-loop turning is inaccurate. This Task 2 platform, scene and skills API are what Task 3 and Task 4 must use.
- v. Provide `Video_Task2` showing the scene with its objects, the onboard camera view, a timed move and a closed-loop turn with the `[TURN]` line visible in the terminal.

Task 3: Typed English Motion Commands via an LLM Parser (Student B)

- i. Research structured-output prompting for robot command parsing (JSON mode, function calling, schema-constrained decoding). Briefly summarize and justify your method in the report, and choose at least two LLM services from Section V.
- ii. Command interface. Define a small JSON command schema (the recommended schema is in Section V: `move` / `turn` / `goto_object` / `stop` / `chat`) and implement an LLM-based parser that converts one free-form English utterance into a list of commands. Multi-step instructions (“move forward for three seconds and then turn back”) must become an ordered sequence; paraphrases (“go straight ahead”, “could you walk forwards a bit”) must be handled; and impossible, unsafe, empty or non-English requests (“fly to the roof”) must be rejected or must trigger a clarification question. Keyword matching alone is not sufficient — an LLM parser is required. Print `[CMD] actions=... n=...` for an accepted command and `[CMD] rejected reason=...` otherwise.
- iii. Chat interface and execution. Implement the multi-turn chat interface in the terminal: the user types a command and confirms with ENTER, the robot executes it and

replies, and the conversation continues. After a valid command is accepted, the robot must commence autonomous motion through the Task 2 skills API — no additional start trigger and no teleoperation — and continue until every action is completed in order. Print [EXEC] and [DONE] lines. The dialogue history must be maintained so that follow-up turns can refer to earlier ones (“do that again, but slower”). Run the chat loop in a separate thread so that the simulation never blocks while waiting for the user or the LLM.

- iv. Evaluate the parser on at least 20 test utterances (including at least 5 paraphrases and 5 invalid/out-of-scope requests) with at least two LLM services: parsing accuracy, latency and cost per call. Present the results in a table in the report and analyze the failure cases.
- v. In Video_Task3, keep the terminal visible. Type the English command so that it is visible in the recording. After ENTER, the robot must start moving on its own and continue until [DONE]. The video must include at least one multi-step command with two or more actions including a turn, and at least one rejected command. State in the report that single-step and multi-step commands are supported, with one example of each.

Task 4: YOLO Object Search and Approach Connected to Task 3 (Student C)

- i. Research object detection and color grounding for robotics (COCO-trained detectors such as YOLO, HSV color classification, open-vocabulary detection). Briefly summarize and justify your method in the report.
- ii. Detection and color grounding. Run YOLO on the Task 2 camera frames and determine each detection's color from the pixels inside its bounding box (e.g., median hue in HSV space). Print [DETECT] class=... color=... conf=... bbox=... lines and show example detections in the report or video.
- iii. Search and approach. Implement the goto_object(class, color) behavior and connect it to the Task 3 interface, so that a typed command such as “go to the green chair” starts autonomous motion exactly as in Task 3. If the target is not visible, rotate in place and re-detect (print [SEARCH]); once detected, steer so that the bounding-box center moves to the image center, walk forward, and stop when the object is found

under (C1)–(C3) below. Report [MISSION] status=FAIL reason=... after a full turn without detection or after a timeout (60 s is a suggested default; state yours in the report). Printing detections without the robot moving, or moving the robot by teleoperation, is not a complete Task 4.

- iv. Evaluate the search and approach on at least 10 trials with different objects and different starting poses, including at least one where the target is not initially visible and at least one same-class disambiguation (green chair vs. red chair): detection accuracy, grounding accuracy (right object chosen), approach success rate, final distance d , and failure analysis. Present the results in a table in the report.
- v. Video_Task4 must keep the terminal visible throughout. Type the English object-search command so that it is visible. The video must show, in order: the typed command, the [CMD] line, the [SEARCH] / [DETECT] lines, the [FOUND] line and the [MISSION] line, for at least two objects, including one that is not initially visible and one same-class disambiguation. In a 2-member group, both members share this task.

Definition of “found” (mandatory for Task 4). An object counts as found only when all of the following hold at the same time:

(C1) Detection: the onboard camera pipeline labels that class and that color from the live camera frame at the moment of stopping.

(C2) Proximity: the planar Euclidean distance from the robot base (trunk) origin to the object center is at most 0.80 m, $d = \sqrt{(x_{\text{base}} - x_{\text{obj}})^2 + (y_{\text{base}} - y_{\text{obj}})^2}$. The z coordinate is ignored.

(C3) Logging: print a one-line message to the terminal, e.g. [FOUND] class=chair color=green t=14.2 s d=0.61 m.

The ground-truth object positions recorded in Task 2 may be used only to compute d for logging and evaluation — never inside the controller to steer the robot. Do not mark an object as found from the ground-truth pose alone. Approaching closer than 0.80 m is allowed but not required; the robot must stop before contact.

Task 5: Videos and report Submission (ALL)

- i. Prepare one group zip (minilab_1.3_group_<index>.zip) that a marker can unpack and reproduce. The report is a group document: list every member, the contribution split, a short self-description of all five tasks, and the AI Usage Declaration. It is not a single-student log and not a generic user manual.
- ii. Submit three videos: Video_Task2 (Task 2 scene, camera view and motion skills, [TURN] visible); Video_Task3 (typed English motion command, fully autonomous execution, [CMD] / [EXEC] / [DONE]); Video_Task4 (typed English object-search command, [CMD] / [SEARCH] / [DETECT] / [FOUND] / [MISSION]). Video_Task3 and Video_Task4 must keep the terminal visible throughout. A video without the required terminal output is incomplete.
- iii. Include the platform and Python version, dependencies, install and run commands, scene-file paths, the LLM service(s) used and how API keys are supplied (never commit keys), the evaluation tables from Tasks 3 and 4, and pointers to the key files owned by Student A, Student B and Student C.

Optional bonus (up to 5 extra marks, capped at 100): speech input (a spoken English command transcribed and passed into the Task 3 interface, shown in a Video_Bonus with [STT] lines); scene description and visual question answering through a VLM (“what can you see?”); replacing YOLO grounding with VLM bounding boxes and comparing the two; navigating to objects placed on one of the rough-terrain maps (e.g., stairs); or multi-goal missions (“visit the ball and then both chairs”).

IV. Score Distributions

In this mini-lab, we will thoroughly assess and assign scores to students based on their performance in three key areas: innovation, usefulness, and teamwork. Specifically, we will evaluate how creatively they approach problem-solving and propose novel ideas (innovation), how practical and applicable their solutions are in addressing real-world challenges (usefulness), and how effectively they collaborate, communicate, and contribute as part of a group (teamwork). Each of these aspects will be carefully considered to provide a comprehensive evaluation of the students' overall contribution to the mini-lab.

The distribution of the scores for each task is listed below.

Groups with 2 or 3 members

The table below is the official score distribution. Task 1 and Task 5 are assessed for the whole group. Task 2, Task 3 and Task 4 are led by Student A, Student B and Student C respectively.

Task	Score Distribution	Name
1	20%	ALL
2	60%	Student A
3	60%	Student B
4	60%	Student C
5	20%	ALL

V. Platform Specification

You are free to choose your robot platform and simulator. The recommended default used in all examples in this document is the example repository: https://github.com/aoqianz/quadruped_mujoco; A Python MuJoCo quadruped simulation with the locomotion problem already solved, so your work is the skills, dialogue, perception and navigation layers on top of it. It runs on Linux (recommended), and the browser control panel also makes it usable over WSL2 on Windows. The repository provides:

Component	What it provides
eg/play.py + eg/model_3400.onnx	Complete keyboard-driven walking demo: pre-trained RL policy (50 Hz) over 200 Hz physics, PD torque control, joint remapping
src/runtime_control/	Browser control panel, live PD/mass/friction tuning, resets, pushes, map switching, three onboard cameras attached to the trunk (dog_front_camera is the first-person view)
src/runtime_control/maps/	11 bundled terrain maps (flat track, stairs, slopes, Google Barkour, rough terrain, dynamic obstacles, ...); your own scene is one more MJCF map registered with MapSpec
eg/dog/	Robot model: MJCF/URDF and meshes

Quick start (example platform):

```
conda create -n quadruped_mujoco python=3.11 -y
conda activate quadruped_mujoco
git clone https://github.com/aoqianz/quadruped_mujoco.git
cd quadruped_mujoco && pip install -e .
pip install mujoco numpy onnxruntime pyyaml pillow ultralytics openai
python eg/play.py # native MuJoCo viewer
python eg/play.py --gui # browser panel at http://localhost:8765
python eg/play.py --headless --duration 1 # quick automated check
```

Keys: W/S forward/back, A/D strafe, Q/E turn, R/F height, T reset

Using your own platform instead is equally welcome — any robot morphology and simulator is acceptable (e.g., another MuJoCo robot such as Unitree Go2/G1 via `unitree_rl_gym`, a wheeled TurtleBot3 in Gazebo/ROS2, Webots, or Isaac Lab) as long as: (a) the simulated robot has an onboard RGB camera whose images you can read programmatically at each control frame; (b) the robot can be commanded to move (velocity commands or equivalent); (c) you can add your own objects to the scene; and (d) locomotion is already solved by an existing package or pre-trained policy — training a walking policy is not part of this mini-lab. The tasks and grading are unchanged regardless of platform. Briefly justify your platform choice in the report, and discuss with the GA before committing to an unusual one.

Velocity command convention (example platform): v_x forward positive, v_y left positive, ω_z turn-left (counter-clockwise) positive, each normalized to $[-1, 1]$; body height in meters (default 0.25 m). Define the robot base as the trunk body origin, x forward, y left, z up. The onboard front camera is rigidly attached to the trunk.

Object detector: use Ultralytics YOLO (e.g., `yolov8n` or `yolo11n`). It is free, downloads once, and runs in real time on a laptop CPU at the resolutions used here. It is pre-trained on the 80 COCO classes — design your scene objects around classes it can actually detect (Task 2.iii). Determine color yourself from the bounding-box pixels; do not expect the detector to know colors.

LLM services (text-only calls suffice; choose at least two for the comparison in Task 3.iv):

Service	Model (example)	Cost / free tier	Note
OpenAI	<code>gpt-5-mini</code> / <code>gpt-5-nano</code>	paid, very cheap (text-only: well under US\$1 for the whole mini-lab)	simplest, good all-round
Alibaba Cloud Model Studio (Singapore region)	<code>qwen-plus</code> / <code>qwen-flash</code>	free quota for new users (≈ 1 M tokens per model, 90 days); enable “free quota only” to avoid billing	OpenAI-compatible API
Google AI Studio	<code>gemini-2.5-flash</code> or newer Flash	free tier, rate-limited — check your live limits in AI Studio	good free option; uses its own google-genai SDK
Ollama (local)	<code>qwen2.5:3b</code> / <code>llama3.2:3b</code>	free, runs on CPU	offline fallback; weaker JSON discipline — test carefully

Recommended interfaces (not mandatory, but strongly recommended so that Task 2 / 3 / 4 code from different members interoperates without rewriting). JSON command schema produced by the Task 3 parser:

Action	Fields	Executed by
move	vx, vy, wz (each in $[-1, 1]$), duration (s)	Task 2 timed-move skill
turn	angle_deg (positive = left)	Task 2 closed-loop turn skill
goto_object	class (COCO name), color	Task 4 search and approach
stop	—	executor (clears the queue)
chat	reply (answer or clarification question)	chat interface (no motion)

Terminal log lines (the videos are graded on these):

Tag	Printed by	Example
[CMD]	Task 3 parser	[CMD] actions=move(vx=0.8, 3.0 s), turn(180 deg) n=2 / [CMD] rejected reason=non-English
[EXEC] / [DONE]	Task 3 executor	[EXEC] action=1/2 move vx=0.8 t=3.0 s / [DONE] actions=2 t=9.4 s
[TURN]	Task 2 turn skill	[TURN] target=180.0 deg final_error=1.8 deg
[DETECT] / [SEARCH]	Task 4 perception	[DETECT] class=chair color=green conf=0.72 bbox=[212,140,318,352]
[FOUND]	Task 4 approach	[FOUND] class=chair color=green t=14.2 s d=0.61 m
[MISSION]	Task 4 approach	[MISSION] status=SUCCESS / [MISSION] status=FAIL reason=timeout

VI. Notes

Task 5 marks the completeness of the project and the three videos.

Video_Task2 must show the Task 2 scene with its objects, the onboard camera view and both motion skills. Video_Task3 is graded on the typed English command, fully autonomous execution and the [CMD] / [EXEC] / [DONE] lines. Video_Task4 is graded on the [DETECT] / [FOUND] / [MISSION] lines and on connecting the typed command to autonomous search and approach.

Technical hints. Run perception at 10–20 Hz, not at the physics rate (200 Hz on the example platform): render the camera every N physics steps. The onboard camera shakes with the gait, so count consecutive misses before declaring an object lost. Keep the LLM out of the fast loop: an API call takes on the order of a second, so the parser runs once per user command in its own thread while the skills and the approach controller run locally. Yaw from the MuJoCo quaternion (w, x, y, z) is $\text{atan2}(2(wz + xy), 1 - 2(y^2 + z^2))$. COCO-trained YOLO does not detect untextured primitives — verify your meshes before building on them. On a headless machine set `MUJOCO_GL=egl`. Never commit API keys; read them from environment variables.

VII. References

- [1] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in Proc. IEEE/RSJ IROS, 2012. <https://mujoco.org>
- [2] Example platform repository: `quadruped_mujoco` — MuJoCo quadruped simulation with runtime control panel, https://github.com/aoqianz/quadruped_mujoco
- [3] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, “Learning to walk in minutes using massively parallel deep reinforcement learning,” in Proc. CoRL, 2021. (`legged_gym`)
- [4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, real-time object detection,” in Proc. IEEE CVPR, 2016.
- [5] Ultralytics, “Ultralytics YOLO documentation,” <https://docs.ultralytics.com>, <https://github.com/ultralytics/ultralytics>
- [6] T.-Y. Lin, et al., “Microsoft COCO: Common objects in context,” in Proc. ECCV, 2014. (the 80 detectable classes)
- [7] M. Ahn, et al., “Do as I can, not as I say: Grounding language in robotic affordances,” in Proc. CoRL, 2022. (`SayCan`)
- [8] J. Liang, et al., “Code as Policies: Language model programs for embodied control,” in Proc. IEEE ICRA, 2023.
- [9] A. Brohan, et al., “RT-2: Vision-language-action models transfer web knowledge to robotic control,” in Proc. CoRL, 2023.
- [10] A. Koubaa, “ROSGPT: Next-generation human-robot interaction with ChatGPT and ROS,” Preprints, 2023. <https://github.com/aniskoubaa/rosgpt>

- [11] NASA Jet Propulsion Laboratory, “ROSA: The Robot Operating System Agent,”
<https://github.com/nasa-jpl/rosa>
- [12] M. Deitke, et al., “Objaverse: A universe of annotated 3D objects,” in Proc. IEEE CVPR, 2023. <https://objaverse.allenai.org> (free object meshes)
- [13] ONNX Runtime, <https://onnxruntime.ai>
- [14] OpenAI, “API documentation,” <https://platform.openai.com/docs>
- [15] Alibaba Cloud, “Model Studio (Singapore region) documentation,”
<https://www.alibabacloud.com/help/en/model-studio/>
- [16] Google, “Gemini API documentation,” <https://ai.google.dev>
- [17] Ollama, <https://ollama.com>